

Enhancing Hangman Solver Performance by Exploring the Game Dynamics

Yu WANG

yuwang91@outlook.com

January 13, 2025

Overview

- 1 Preview of Results
- 2 Hangman Game Dynamics
- 3 Game-Dynamics-Aware Hangman Solver
- 4 Weighted N-grams Model
- 5 Information Entropy Model
- 6 Fine-tuned BERT Model
- 7 Other Implementation Strategies
- 8 Results and Analysis
- 9 Appendix for Additional Analysis

Outline

- 1 Preview of Results
- 2 Hangman Game Dynamics
- 3 Game-Dynamics-Aware Hangman Solver
- 4 Weighted N-grams Model
- 5 Information Entropy Model
- 6 Fine-tuned BERT Model
- 7 Other Implementation Strategies
- 8 Results and Analysis
- 9 Appendix for Additional Analysis

Best Result

Final method:

Fine-tuned BERT method, featured with:

- Custom tokenizer
- Iterative rollback

Best result for local tests on local, disjoint testing dataset:

- Success rate = 74.4%
- Time overhead = 20 min 13 sec for 1,000 games (on 1 NVIDIA 4090)
- Local test set = English word list \ training set

Result for API simulation submission:

- Success rate = 65.5%

Outline

- 1 Preview of Results
- 2 Hangman Game Dynamics**
- 3 Game-Dynamics-Aware Hangman Solver
- 4 Weighted N-grams Model
- 5 Information Entropy Model
- 6 Fine-tuned BERT Model
- 7 Other Implementation Strategies
- 8 Results and Analysis
- 9 Appendix for Additional Analysis

Progression in Hangman Game

The hangman game progresses from two perspectives:

- **Revealed Letters:**

- As the percent or number of revealed letters increase (up to 100% or `len(Word)`):
 - More contextual information for next guess.

- **Failed Attempts:**

- As failed attempts increase (up to 6):
 - Higher risk of losing the game.

Information Dynamics with Game Progression

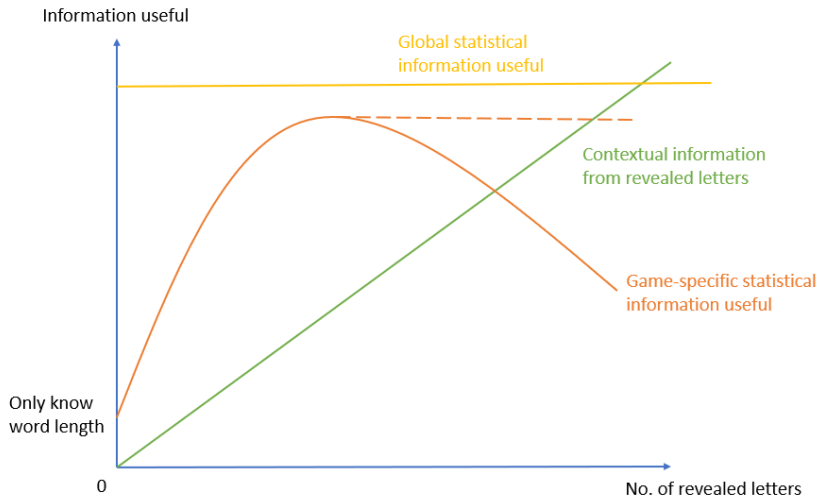


Figure: Information Dynamics w.r.t. No. of Revealed Letters

Information Dynamics with Game Progression

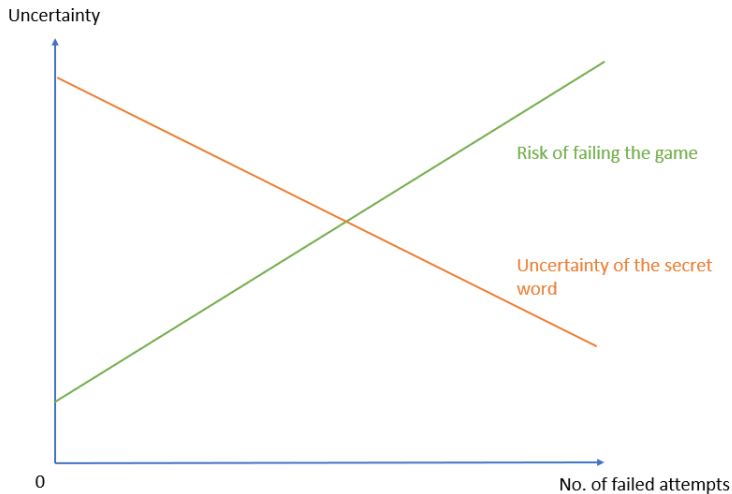


Figure: Information Dynamics w.r.t. No. of Failed Attempts

Outline

- 1 Preview of Results
- 2 Hangman Game Dynamics
- 3 Game-Dynamics-Aware Hangman Solver**
- 4 Weighted N-grams Model
- 5 Information Entropy Model
- 6 Fine-tuned BERT Model
- 7 Other Implementation Strategies
- 8 Results and Analysis
- 9 Appendix for Additional Analysis

Definitions of the “Best” Letter to Guess

Condition on current game state:

- **“Most Frequent”:**
 - The most probable letter across the entire dictionary.
- **“Most Informative”:**
 - The letter that provides the most information gain.
- **“Most Contextualized”:**
 - The most context-relevant letter (based on English morphology and lexical knowledge).

Information Dynamics and Dynamic Strategy

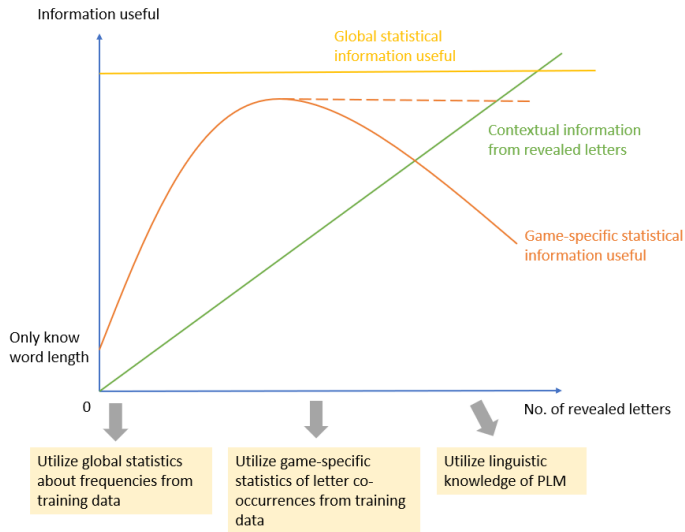


Figure: Information Dynamics w.r.t. No. of Revealed Letters

Information Dynamics and Dynamic Strategy

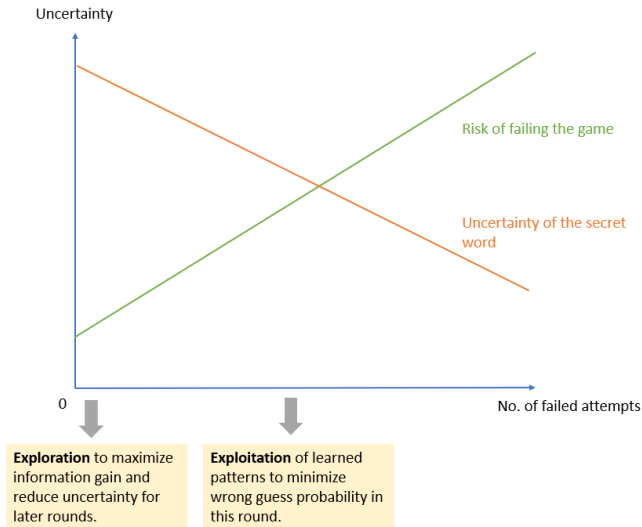


Figure: Information Dynamics w.r.t. No. of Failed Attempts

Three Phases of a Hangman Game

- ① **Opening** (e.g., no or few failed attempts)
 - Little to no context information available, requiring **exploration**.
 - Focus on maximizing **information gain** to prepare for later rounds.
- ② **Midgame** (e.g., 20-80% letters revealed)
 - Sufficient context available to leverage statistical patterns.
 - Use global and local statistical information about **letter co-occurrences** as **exploitation**.
- ③ **Endgame** (e.g., ≤ 2 remaining letters)
 - Sufficient context available to leverage linguistic knowledge.
 - Utilize **linguistic knowledge** (e.g., morphology and lexical prior knowledge).

Three Phases of a Hangman Game

- ① **Opening** (e.g., no or few failed attempts)
 - Information gain maximization objective.
 - **Information entropy** method.
 - Guess the “**most informative**” letter.
- ② **Midgame** (e.g., 20-80% letters revealed)
 - Risk minimization objective.
 - **Weighted n-grams** method.
 - Guess the “**most frequent**” letter.
- ③ **Endgame** (e.g., ≤ 2 remaining letters)
 - Risk minimization objective.
 - **Pre-trained LM** (e.g., fine-tuned BERT) method.
 - Guess the “**most contextualized**” letter.

Game-Dynamics-Aware Hangman Solver

Algorithm GameDynamicsAwareHangmanSolver

Require: word_length, known_letters, incorrect_guesses

Ensure: next_guess

```
1: count(unknown_slots)  $\leftarrow$  word_length - count(known_letters)
2: if count(known_letters)  $< \alpha$  and count(unknown_slots)  $> \beta$  and
   count(incorrect_guesses)  $< \gamma$  then
3:   next_guess  $\leftarrow$  SELECTINFORMATIONENTROPYMODEL()
4:   return next_guess
5: else if count(unknown_slots)  $> \theta$  then
6:   next_guess  $\leftarrow$  SELECTWEIGHTEDNGRAMSMODEL()
7:   return next_guess
8: else
9:   next_guess  $\leftarrow$  SELECTFINETUNEDBERTMODEL()
10:  return next_guess
11: end if
```

Dynamic Weighting as an Extension

- 1 Determine the **weights** of each model **dynamically based on the game phase**.
- 2 Combine the three “probabilities” using a weighted sum:

$$P = w_1 \cdot P_{\text{entropy}} + w_2 \cdot P_{\text{ngrams}} + w_3 \cdot P_{\text{bert}}$$

where w_1, w_2, w_3 are dynamic weights dependent on game states, and P is final metric of choosing a specific letter.

- Optimal weights found by grid search.
 - Entropy values normalized to be in $[0, 1]$.
- 3 Choose the letter k^* with the highest metric:

$$k^* = \arg \max_k P(k)$$

Outline

- 1 Preview of Results
- 2 Hangman Game Dynamics
- 3 Game-Dynamics-Aware Hangman Solver
- 4 Weighted N-grams Model**
- 5 Information Entropy Model
- 6 Fine-tuned BERT Model
- 7 Other Implementation Strategies
- 8 Results and Analysis
- 9 Appendix for Additional Analysis

Motivation of Weighted N-Grams Model

Observation and motivation

- Certain letters are more likely to appear together as **spelling patterns** in English (e.g., “ed”, “th”, “ing”).

Given the current game state,

- Select the most probable letter based on the frequencies of different **matched patterns** from training data.

Step 1: Build N-Gram Dictionary

N-Grams:

- N-length spelling patterns.
- Padded with # (start) and \$ (end) to add boundary information.

Example:

- Word: hello $\rightarrow$ Padded: #hello\$
- 2-gram: {#h, he, el, ll, lo, o\$}
- 3-gram: {#he, hel, ell, llo, lo\$}

Frequency Count:

- Corpus (i.e., training set): {hello, world}
- 2-grams of corpus: he: 1 (from hello), wo: 1 (from world)

Result:

- A dictionary of all n-grams and their frequencies, e.g.,

n-grams = {1 : {h: 1, e: 1, o: 2}, 2 : {he: 1, wo: 1}, ...}

Step 2: Calculate N-Gram Probabilities

Goal: Compute the probability of each letter for the unknown slot(s).

- 1 Identify substrings in the secret word with unknown slot(s) (“.”).
- 2 Match these substrings with n-grams in the dictionary.
- 3 Calculate weighted probabilities for each possible letter.

Example:

- Secret Word: h.11o
- Candidate Substrings (Ignoring Padding):
 - 2-gram: h., .1, 11, 1o
 - 3-gram: h.1, .11, 11o

Unknown Slots:

- n from 1 to 3:
 - 1 unknown slots
- n from 4 to 6:
 - 1 or 2 unknown slots

Example:

- Candidate Substring: fully
 - 1 unknowns: .ully, f.lly, fu.ly, ful.y, full.
 - 2 unknowns: ..lly, .u.ly, .ul.y, etc.

Step 2: Calculate N-Gram Probabilities

Weighted Contribution:

$$P(\text{letter}) = \sum_{n=1}^6 w_n \cdot P_n(\text{letter})$$

where

- n : size n of n-grams.
- $P_n(\text{letter})$: $\frac{\text{counts of n-grams with this letter in unknown slots}}{\text{total counts of n-grams with each of 26 letters in unknown slots}}$ learned from the training data.
- w_n : weight for n-grams of size n .

Step 3: Select the Best Letter

Goal: Use the weighted probabilities to determine the best letter to guess.

- 1 Normalize probabilities (summing to 1).
- 2 Exclude already guessed letters.
- 3 Sort letters by probabilities.

Example:

- Word: h.11lo
- Normalized Probabilities:
 - a: 0.1, e: 0.3, i: 0.2, o: 0.4
- Already Guessed: {a, e}
- Best Guess: o (highest probability).

Outline

- 1 Preview of Results
- 2 Hangman Game Dynamics
- 3 Game-Dynamics-Aware Hangman Solver
- 4 Weighted N-grams Model
- 5 Information Entropy Model**
- 6 Fine-tuned BERT Model
- 7 Other Implementation Strategies
- 8 Results and Analysis
- 9 Appendix for Additional Analysis

Motivation of Information Entropy Model

Observation and motivation

- Not all frequent letters are equally valuable to guess:
 - e.g., both “s” and “e” are frequent letters in words, but their **frequent word positions** are different.

Given the current game state,

- Select the letter that can **reduce the search space** more (i.e., highest information gain).

Information Gain and Entropy

Information gain measures the reduction in uncertainty (entropy) after a certain event (i.e., guessing a letter):

- **Information gain from guessing letter i** is the reduction in entropy (reduced to 0) by knowing:
 - Whether the letter is in the word
 - The position(s) of the letter

Entropy H of a random variable:

$$H = - \sum_i p_i \log_2(p_i)$$

where p_i is the probability of the i -th outcome.

Step 1: Filter Candidate Words

Goal: Narrow down the list of possible words using the current game state.

Example:

- Word Pattern: b.n.n.
- Dictionary: {banana, bonbon, cannon, beacon, ...}
- Filtered Candidates: {banana, bonbon}

Implementation:

- 1 Filter candidate words in the dictionary (i.e., the training set $\star$) by pattern.
- 2 Identify unguessed letters.
- 3 Count frequencies of binary patterns for each letter
 - e.g., “1” if position = letter, “0” otherwise

Step 2: Calculate Entropy for Each Letter

Goal: Compute the information gain (entropy) for each unguessed letter.

$$H = - \sum_i p_i \log(p_i)$$

where:

- p_i : probability of observing pattern i when guessing a letter.

Example:

- **Filtered Candidates:** {banana, bonbon, binbin}
- **Letter to Guess:** a
- **Outcome Patterns:**
 - Pattern "010101" (e.g., "banana"): $p = \frac{1}{3}$
 - Pattern "000000" (e.g., "bonbon", "binbin"): $p = \frac{2}{3}$
- **Entropy:**

$$H = - \left(\frac{1}{3} \log \frac{1}{3} + \frac{2}{3} \log \frac{2}{3} \right)$$

Why Information Entropy Method Fails in This Project

Problem: Training and testing sets are **disjoint** in this project.

- Entropy calculations rely on a candidate word list derived from the dictionary.
 - The secret word (from the test set) cannot be in the candidate word list (based on the training set).
 - Entropy estimates of each candidate letter are misleading.

If **training set** = **testing set**, information entropy method will be:

- Useful for game opening.
- Useful for games with more failed attempts allowed.

Outline

- 1 Preview of Results
- 2 Hangman Game Dynamics
- 3 Game-Dynamics-Aware Hangman Solver
- 4 Weighted N-grams Model
- 5 Information Entropy Model
- 6 Fine-tuned BERT Model**
- 7 Other Implementation Strategies
- 8 Results and Analysis
- 9 Appendix for Additional Analysis

Motivation of Fine-tuned BERT Model

Observation and motivation

- Hangman game rule looks like **masked language modeling (MLM)** of training BERT.
- BERT is a PLM with **prior knowledge** of English words.

Given the current game state,

- Ask a fine-tuned BERT model to guess the unknown slots (i.e., letters replaced by [MASK] token).

Step 1: Tokenizer Preparation

Version 1: **Extended BERT Tokenizer**

- Based on the pre-trained BERT tokenizer.
- Extended vocabulary to include 26 letters (a-z).
- **Example:**
 - Input: computer
 - Tokens: [CLS] c o m p u t e r [SEP]
 - Token IDs: [101, 30599, 30596, 30597, 30593, 30595, 30594, 30599, 30592, 102]

Step 1: Tokenizer Preparation

Version 2: **Custom Tokenizer**

- BERT originally uses **subword tokenization** (e.g., com/put/er).
- Probably a **character-level tokenizer** is better for hangman game.
- Vocabulary:
 - Special Tokens: [PAD], [UNK], [CLS], [SEP], [MASK]
 - 26 Letters: (a-z)
- **Example:**
 - Input: computer
 - Tokens: [CLS] c o m p u t e r [SEP]
 - Token IDs: [3, 15, 14, 13, 16, 21, 20, 5, 18, 4]

Step 2: Dataset Preparation for MLM

Randomly mask **only 1 or 2 letters** in each word in the training data.

Example:

- Word: computer
- Masked Input: [CLS] c o m [MASK] u t e r [SEP]
- Labels: -100 -100 -100 -100 p -100 -100 -100 -100
- Padding: up to fixed length of 18 with [PAD]
- Attention Mask: 1 1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0

Step 3: Training Configurations

- Objective: Minimize cross-entropy loss at masked positions.
- Optimizer: AdamW with learning rate $5e-5$.
- Scheduler: Linear decay scheduler.
- Early Stopping:
 - Monitor validation loss.
 - Stop training if no improvement after 3 epochs.
- Epochs: Maximum 20.
- Training set size: Triple of the given set (w/o data preprocessing).

Step 4: Model Inference

- ① Prepare current game state:
 - Add [CLS] and [SEP] tokens to the word.
 - Replace unknown slots (e.g., ".") with [MASK] tokens.
- ② Pad the word:
 - Pad to a fixed length (e.g., 32 for long words).
 - Create an attention mask.
- ③ Pass the input to the model:
 - Obtain logits for the masked tokens.
- ④ Process logits:
 - Extract logits corresponding to the masked positions.
 - Convert logits to probabilities using softmax.
 - Sum probabilities over masked positions.
- ⑤ Adjust probabilities:
 - Set those of already guessed letters to zero.
- ⑥ Predict the best letter:
 - Select the letter with highest adjusted probability.

Comparisons of Training Tokenizers

Extended BERT Tokenizer:

- Utilizes **pre-trained subword embeddings**
 - Subword tokenization (e.g., com/put/er) resembles spelling patterns.
 - Pre-trained subword knowledge helps with generalization on disjoint test set.

Custom Tokenizer:

- Learns direct **character-level mapping**
 - Learns “[MASK] = one missing letter” without ambiguity.
 - Aligns better with Hangman game rules.

Potential Improvements

Pretrained Model

- 1 Use a character-level model for character-level input (e.g., charBERT, characterBERT)

Masking Method

- 1 Keep a small percentage of words unmasked (i.e., same as vanilla BERT)
 - “To bias the representation towards the actual observed word”
- 2 Mask all occurrences of the same letter in a word
 - To align with Hangman game rules

Other Aspects

- 1 Use different model sizes (e.g., BERT-large)
- 2 Size of custom training data

Outline

- 1 Preview of Results
- 2 Hangman Game Dynamics
- 3 Game-Dynamics-Aware Hangman Solver
- 4 Weighted N-grams Model
- 5 Information Entropy Model
- 6 Fine-tuned BERT Model
- 7 Other Implementation Strategies**
- 8 Results and Analysis
- 9 Appendix for Additional Analysis

Iterative Rollback Strategy

Given that the training set and testing set are **disjoint**:

- ① When **two letters** remain unguessed:
 - Fill the secret word with guessed letter.
 - Calculate normalized **similarity** between the filled word and all words in training set.
 - If exceed a threshold (e.g., 0.95) for any training word, rollback and try the next best letter.
- ② When **one letter** remains unguessed:
 - Fill the secret word with guessed letters.
 - Check if the filled word is in the training set.
 - If so, rollback and try the next best candidate letter.

The rollback strategy is iterative.

Preprocessing Strategy

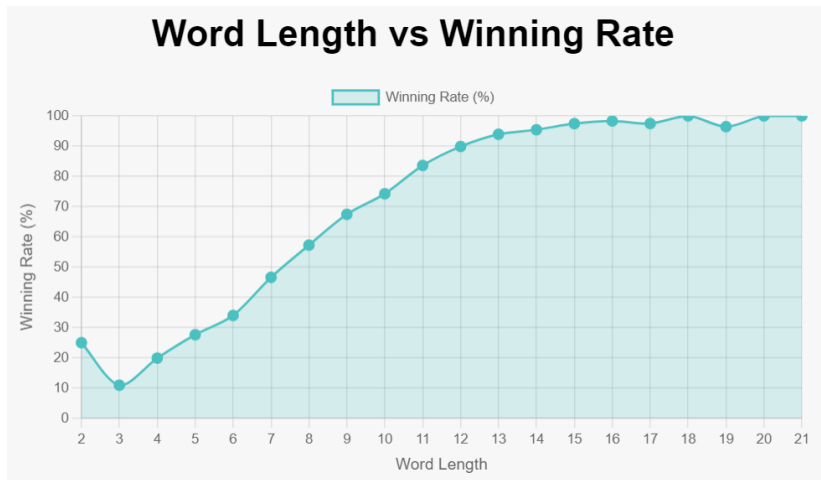
1. Remove words containing only one kind of letter (e.g., “kkkkk”):
 - Such patterns are not helpful.
 - Training set and test set are disjoint.
2. Remove very short words (e.g., $\text{len}(\text{Word}) < 4$):
 - Avoid overfitting to the training data for short words.
 - Training set and test set are disjoint.
2. Remove very long words (e.g., $\text{len}(\text{Word}) > 25$):
 - Facilitate padding and avoid unforeseen errors.

Outline

- 1 Preview of Results
- 2 Hangman Game Dynamics
- 3 Game-Dynamics-Aware Hangman Solver
- 4 Weighted N-grams Model
- 5 Information Entropy Model
- 6 Fine-tuned BERT Model
- 7 Other Implementation Strategies
- 8 Results and Analysis**
- 9 Appendix for Additional Analysis

Success Rate of Only Using Weighted N-grams

Success rate of weighted n-gram method by $\text{len}(\text{Word})$:



Sample Results of Fine-tuned BERT

Input Word: **califragilisticexpialidocious**

Masking Indices: [17] (Masked Characters: ['p'])

Guessed Correct (Revealed Letters): {'a', 'e', 't', 'f', 'l', 's', 'g', 'd', 'c', 'o', 'u', 'r', 'x', 'i'}

Probability Distribution (sorted by value):

y: 0.7075
h: 0.0040
q: 0.0022
p: 0.0016
m: 0.0001

...

Predicted Letter: **y (Probability: 0.7075)**

Test Failed! Guessed: y, Expected: ['p']

Input Word: **dog**

Masking Indices: [0, 2] (Masked Characters: ['d', 'g'])

Guessed Correct (Revealed Letters): {'o'}

Probability Distribution (sorted by value):

t: 0.0650
s: 0.0633
p: 0.0611
b: 0.0601
c: 0.0600

...

Predicted Letter: **t (Probability: 0.0650)**

Test Failed! Guessed: t, Expected: ['d', 'g']

Figure: Examples of Predictions of Masked Characters by Fine-tuned BERT (Left: 1 Unknown Letter, Right: 2 Unknown Letters)

Sample Results of Fine-tuned BERT

Input Word: pseudopseudohypoparathyroidism

Masking Indices: [17, 19, 20] (Masked Characters: ['a', 'a', 't'])

Guessed Correct (Revealed Letters): {'i', 'h', 'e', 's', 'm', 'd', 'p', 'o', 'u', 'r', 'y'}

Probability Distribution (sorted by value):

t: 0.7973
c: 0.0829
a: 0.0731
x: 0.0196
n: 0.0108
...

Predicted Letter: t (Probability: 0.7973)

Test Passed! Guessed: t, Expected: ['a', 'a', 't']

Input Word: engineering

Masking Indices: [2, 3, 7, 8, 10] (Masked Characters: ['g', 'i', 'r', 'i', 'g'])

Guessed Correct (Revealed Letters): {'e', 'n'}

Probability Distribution (sorted by value):

g: 0.3504
i: 0.3475
r: 0.1848
k: 0.0203
t: 0.0192
...

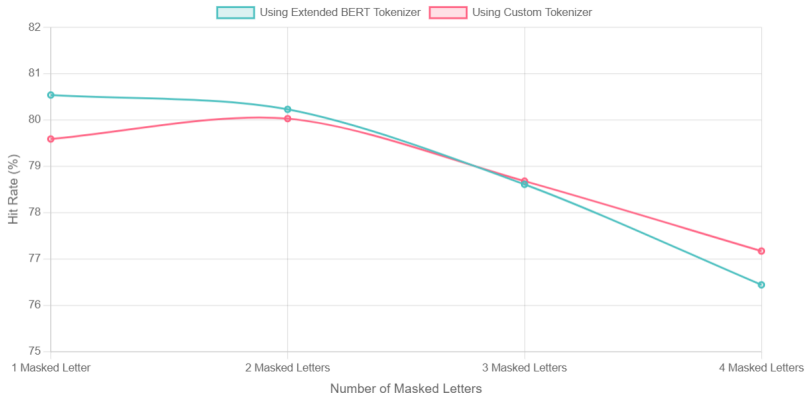
Predicted Letter: g (Probability: 0.3504)

Test Passed! Guessed: g, Expected: ['g', 'i', 'r', 'i', 'g']

Figure: Example of Predictions of Masked Characters by Fine-tuned BERT (Left: 3 Unknown Letters, Right: 4 Unknown Letters)

Hit Rates of Fine-tuned BERT

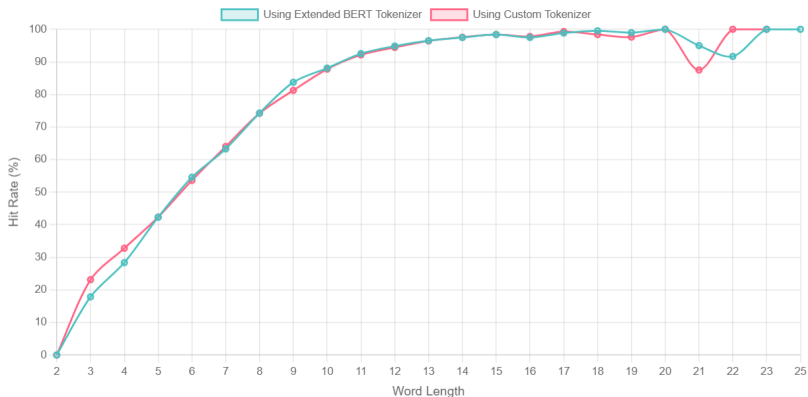
Rate of Correct Guesses (Hit Rate) by Number of Masked Letters



Note: These results have not been post-processed to exclude incorrectly guessed letters. If excluded, the hit rate will improve.

Hit Rates of Fine-tuned BERT

Rate of Correct Guesses (Hit Rate) by Word Length

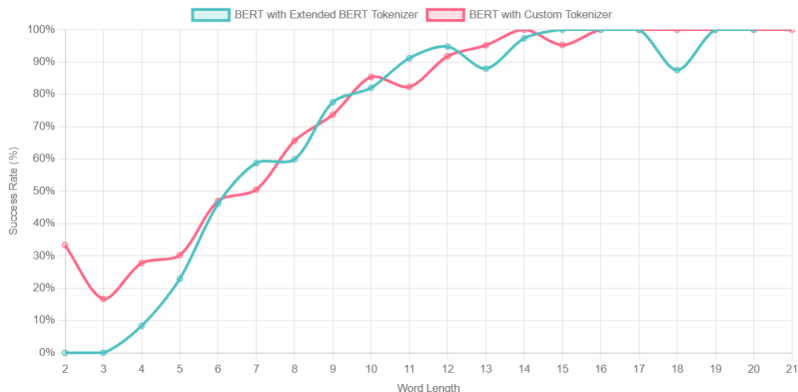


Note: These results have not been post-processed to exclude incorrectly guessed letters. If excluded, the hit rate will improve.

Success Rate of Only Using Fine-tuned BERT

Success rate of (1) BERT with extended BERT tokenizer, and (2) BERT with custom tokenizer by $\text{len}(\text{Word})$:

Success Rate by Word Length



Estimated Average Effect on Success Rate

Baseline Model: Weighted N-Grams (N=5) on Disjoint Data Sets

Description	Sign	Effect on Success Rate
Preprocessing of Training Data	+	0%-1%
N=4 in N-Grams	-	12%
N=3 in N-Grams	-	35+%
Two Unknowns in N-Grams	+	3%-4%
Weights of N-Grams		30+%
Non-Cumulative Frequency Used	+	2%
Padding for Word Boundary and N=6 in N-Grams	+	7%
Rollback Enabled	+	0%-1%
Similarity Threshold for Rollback = 0.975	+	1%
Disjoint Testing and Training Sets		3%-4%
Info Entropy for Opening with Same Testing and Training Sets	+	1%-2%
BERT for Endgame	+	3%-4%
BERT for Entire Game	+	6%

“**BERT for Entire Game**” has (slightly) higher success rate than:

- Game-Dynamics-Aware Strategy
 - Weighted n-grams for opening, BERT for endgame.
- “N-grams for Entire Game”

Hypotheses about its superiority:

- Learn prior linguistic knowledge of **test set** from **pre-training**
- Learn hangman **game rules** and spelling patterns of **training set** from **fine-tuning**

Best Result

Final method:

Fine-tuned BERT method, featured with:

- Custom tokenizer
- Iterative rollback

Best result for local tests on local, disjoint testing dataset:

- Success rate = 74.4%
- Time overhead = 20 min 13 sec for 1,000 games (on 1 NVIDIA 4090)
- Local test set = English word list \ training set

Result for API simulation submission:

- Success rate = 65.5%

References



David Atkins.

Solving hangman with information theory, October 2022.

Exploration of using information theory to optimize hangman gameplay.



Nick Berry.

A better hangman strategy, April 2012.

Analysis of letter frequencies and strategies for improving hangman gameplay.



icefox.

A better strategy for hangman, March 2012.

Discussion on using information gain to improve hangman strategies.



Zhiwei Zhang.

Hangman-game-solvers: Nlp and deep learning solvers for hangman game, 2024.

Repository implementing NLP and deep learning approaches to solving Hangman.

Appendix

Outline

- 1 Preview of Results
- 2 Hangman Game Dynamics
- 3 Game-Dynamics-Aware Hangman Solver
- 4 Weighted N-grams Model
- 5 Information Entropy Model
- 6 Fine-tuned BERT Model
- 7 Other Implementation Strategies
- 8 Results and Analysis
- 9 Appendix for Additional Analysis

Alternative Definition of “Most Frequent”

Cumulative frequency:

- Select the most cumulative letter in the matched n-grams
 - Counting multiple appearances of the same letter

Non-cumulative frequency:

- Select the most frequent letter in the matched n-grams
 - Counting multiple appearances of the same letter as once.
 - Aligned with hangman game rule, i.e., a correct guess reveals all occurrences of the guessed letter.
 - Empirically better.
 - Used in my solution.

Tradeoff Between Information Gain and Risk

- **Information Gain:**

- Guess the letter with the highest information gain in the current round.
- Tradeoff:
 - Minimize search space for next round.
 - Higher probability of a wrong guess, which reduces lives.

- **Risk:**

- Guess the letter with the highest prior probability in the current round.
- Tradeoff:
 - Minimize risk of penalty for a wrong guess.
 - (Probably) larger search space in next round.

Abandoned Preprocessing Strategy

1. Add the trailing “s” for nouns w/o trailing “s”, and remove the trailing “s” for nouns w/ trailing “s”:
 - If the train-test split is random, this operation can convert the noun forms in training data to noun forms in testing data.
2. Add the trailing “s” for verbs w/o trailing “s”, and remove the trailing “s” for verbs w/ trailing “s”:
 - If the train-test split is random, this operation can convert the verb forms in training data to verb forms in testing data.

Abandoned Preprocessing Strategy

The first two preprocessing strategy can be used when all of the following hypotheses hold:

- Each noun has both singular and plural forms.
- Each verb has both third-person singular and plural forms.
- The training and test sets are randomly split.

However, the word form analysis show that they do not hold:

Nouns:

Total nouns: 93538

Singular nouns: 69043

Plural nouns: 24495

Plural nouns with trailing 's': 22087

Verbs:

Total verbs: 55982

Base form verbs: 22955

Third-person singular verbs: 3612

Third-person singular verbs with trailing 's': 3377

Past and past participle verbs: 17249

Present participle verbs: 11340

Figure: Word Form Analysis for Training Data

Comparisons with Zhang's Solution

My weighted n-grams model improves Zhang's solution in terms of:

- ➊ Add padding to indicate word boundary;
- ➋ Extend to 6-grams;
- ➌ Calculate non-cumulative frequency.

Value of a Correct Guess

A **correct** guess is usually more valuable than an “**informative**” guess because it provides:

- ① Whether the letter exists in the word;
- ② How many times the letter occurs in the word;
- ③ The exact positions of the letter in the word.

Moreover, there is zero cost or penalty for a correct guess.

Word Length vs. Number of Unique Letters

Two definitions of “word length”:

- ① $\text{len}(\text{Word})$
 - Known to the player at the start
- ② Number of unique letters in the word
 - Unknown to the player at the start

The number of unique letters is more important:

- Words with fewer unique letters are harder to guess.
- There is less room for “exploration”.

Word Length Distribution

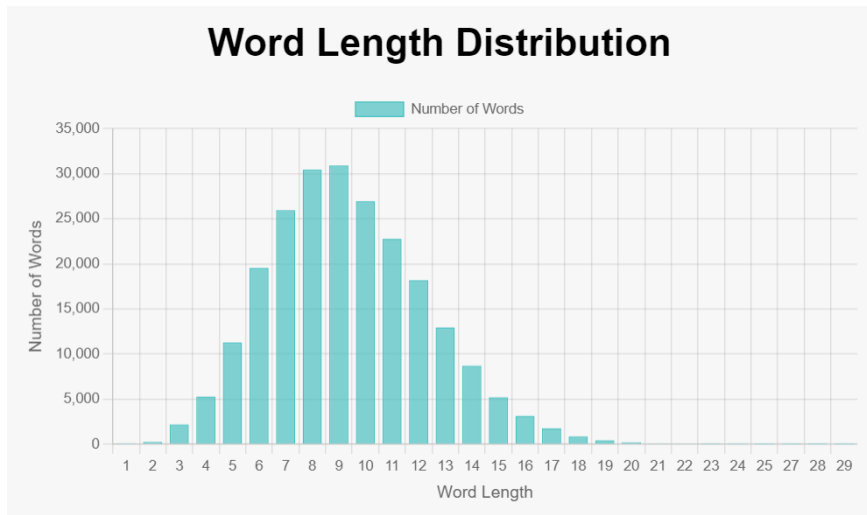


Figure: Word Length Distribution of Training Data

Word Length and Game Difficulty

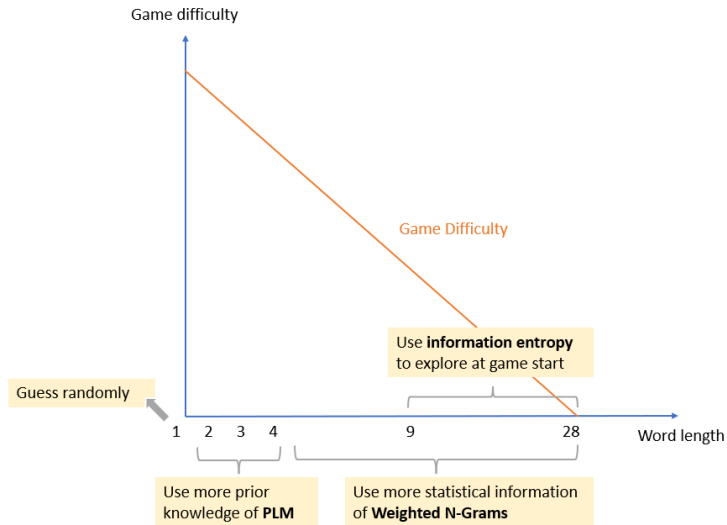


Figure: Word Length, Game Difficulty, and Preferred Method